

Zach Jacokes
5/5/2023
DS6210 Final Project

Before developing any models or optimization algorithms, standard preprocessing was performed as part of the given starter code, which included shuffling the indices, flattening the three-dimensional images into 12288 pixel vectors, and one-hot encoding the labels. I split the 100,000-image sample using an 80-20 train-test ratio. I also decided to normalize the image pixel values, but I wanted to make sure this was actually achieving something, so I ran most of my models both with and without normalization. Zero-one normalization appeared to be the norm for image classification analysis [1], [2], but after discussion with Dr. Baek during office hours, I adjusted the normalization to make the pixel values between negative one and one. Zero-centered normalization is more effective for certain initialization methods like He initialization because it makes the input data more normally distributed [3]–[6]. This way the network learns more efficiently. The zero-one normalization was done by dividing the flattened X_{train} variable by 255, and the zero-centered normalization was done by dividing the X_{train} variable by 128, then subtracting one. The last preprocessing step I took was to randomly select a smaller subset of both the train and test set to run my models; I wanted to be sure they would work before analyzing the entire dataset. The training subset had 10,000 images, and the test subset had 2,000 images.

To achieve a baseline for this project, I started with a vanilla implementation of logistic regression. This implementation utilized stochastic gradient descent, softmax activation, and mini-batching, and I tested it using a variety of hyper parameters, most notably adjusting the learning rate and number of epochs. I found that, while the model appeared to be learning the features of the training set, the test set accuracy and loss remained relatively constant throughout. When I tested this model with the normalized input data, the test accuracy and loss began to improve, but not by much; convergence appeared to happen much slower as evidenced by the much lower test and train accuracy. Interestingly, the normalized input iteration showed much smoother loss and accuracy curves, which is likely due to the uniformity of the normalized data. I expected this model to perform poorly, but I was encouraged by the initial results so I decided to continue with logistic regression to see if I could improve it. Results from the first modeling attempt are shown in Figure 1.

The next model I tried was logistic regression with a momentum term. I incorporated momentum in an effort to improve the test accuracy, as it appeared the model was finding local minima in the test set. Momentum helps avoid local minima because it incorporates prior gradient values, thus allowing the model to “overshoot” local minima in the pursuit of the overall trend of the loss function [7]. I still used gradient descent, softmax activation, mini-batching, and several sets of hyper parameters. The results from the non-normalized inputs look similar to the previous model without momentum in that the accuracy and loss curves are choppy, and the normalized input results look similarly smooth. The training accuracy for the normalized data was much higher than the model without momentum, which indicates that the momentum term was somewhat effective. During this trial, I noticed some warnings about encountering overflow in the softmax function, so I incorporated a simple clipping procedure softmax. This way I was able to improve the numerical stability of the softmax function by limiting the values passed to the denominator. The results of this model are shown in Figure 2.

The last model I tried was a 3-layer neural net. I abandoned the piece-wise approach I had planned to use in the interest of getting some positive results, so I incorporated several new techniques into this model. The first was upgrading the momentum term to Adam; this technique is an improvement on naïve momentum because it adapts the learning rate based on each parameter individually, leading to more specifically-tailored weight updates [8]. The second improvement I included in this model was L2 regularization. I was consistently seeing overfitting on the training dataset, and regularization helps to reduce overfitting and improve generalizability by punishing the complexity of the model [9]. The last improvement I made was to use He initialization, which initializes the model weights based on the variable input sizes for each layer. This way we maintain consistent variance for activation throughout the network. I toyed with the hidden layer sizes some to attempt to improve the accuracy, and found that sizes of 512, 256, and 128 were the best. The results from this model were more promising: as shown in Figure 3, the test accuracy was far higher than the logistic regression attempts, though we are still likely hitting local minima instead of continuing to follow the gradient down to a global minimum.

Overall, all of the improvements I made to my modeling efforts appear to have helped despite not being able to break through 10% testing accuracy. In particular, I believe the He initialization and Adam helped the most in improving my results. Though it did not boost my accuracy much, there was a noticeable improvement when I attempted to train my models on the full 100,000 image dataset as opposed to the 10,000 image subset. There is a chance that my models would have converged if I had been able and willing to run them with a smaller learning rate for many more epochs, as evidenced by the consistent training loss decreases, but the marginal cost of doing so (time) was not worth the small chance of convergence given my time constraint. It is encouraging that my models were in fact optimizing at all, and with more time I believe I could achieve closer to 20% test accuracy. Of interest is the differential loss values between the different models: the logistic regression models had loss values starting close to 18, whereas the neural net loss values started closer to 6. I believe we can attribute this disparity to the ability of the neural net to capture more complexity in the features, though I may have found lower initial loss values in my logistic regression models if I had used He initialization to initialize the weights.

I experienced several issues when attempting to build models. The first is that I was unable to get any Newton or quasi-Newton method to run on my computer or on Google Colab; the memory-intensive nature of these methods would crash my local machine or send me far over the limit allowed on Colab. I did try to move my modeling efforts to Rivanna, but I could not get the dataset to download there, likely due to security reasons. I could have uploaded the flattened vectors of input data to Rivanna and tried testing from there, but I was short on time and decided to focus more on the bonus implementation opportunities. Another model I tried and failed to implement was a convolutional neural net; I figured a CNN would likely perform better on image data since it looks at “windows” of input data at a time to account for spatially-dependent features. However, implementing it from scratch proved to be too difficult and time-consuming so I made the decision to focus on improving my ANN instead.

Figures

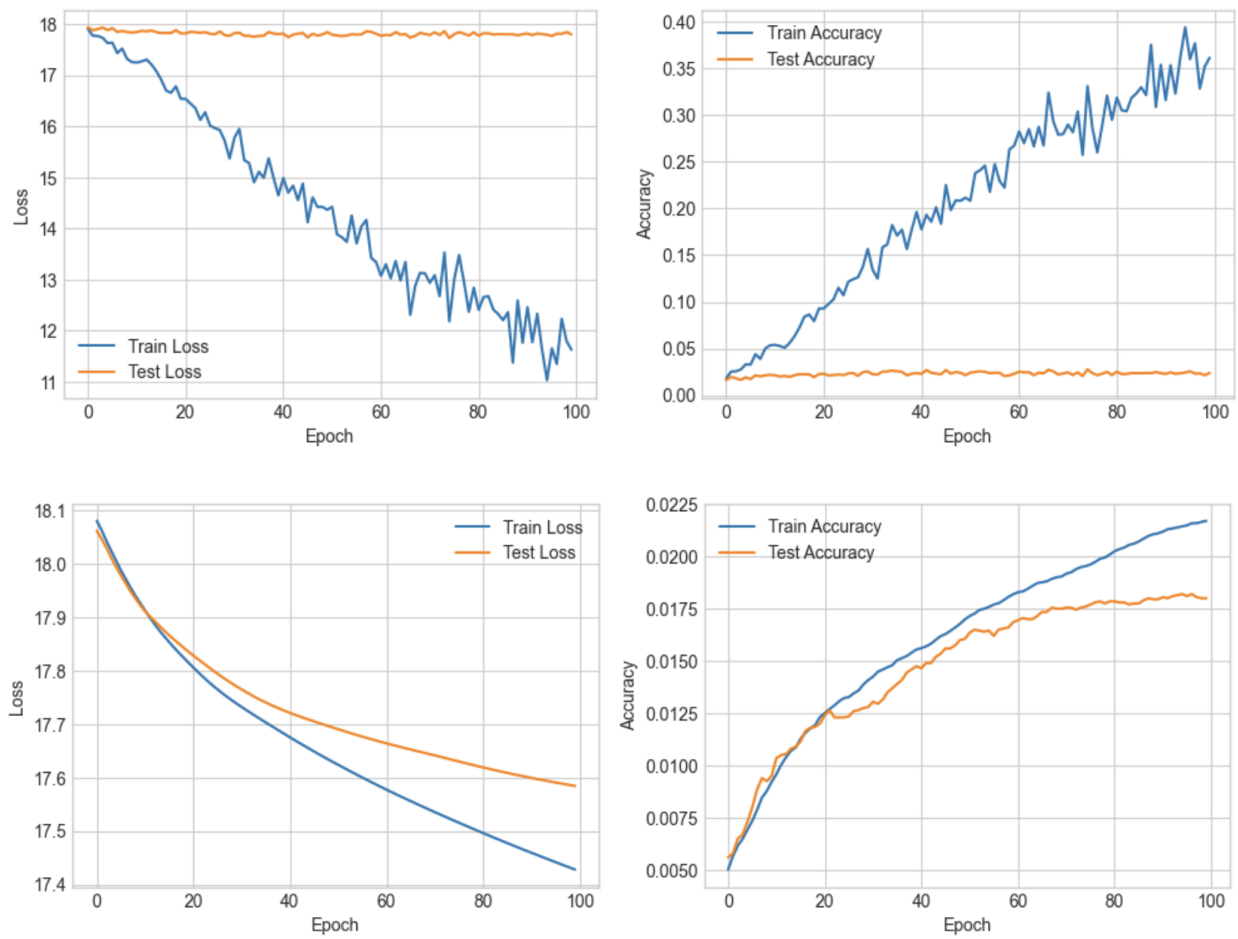


Figure 1. Top: Vanilla logistic regression with non-normalized input data. Bottom: Vanilla logistic regression with normalized input data.

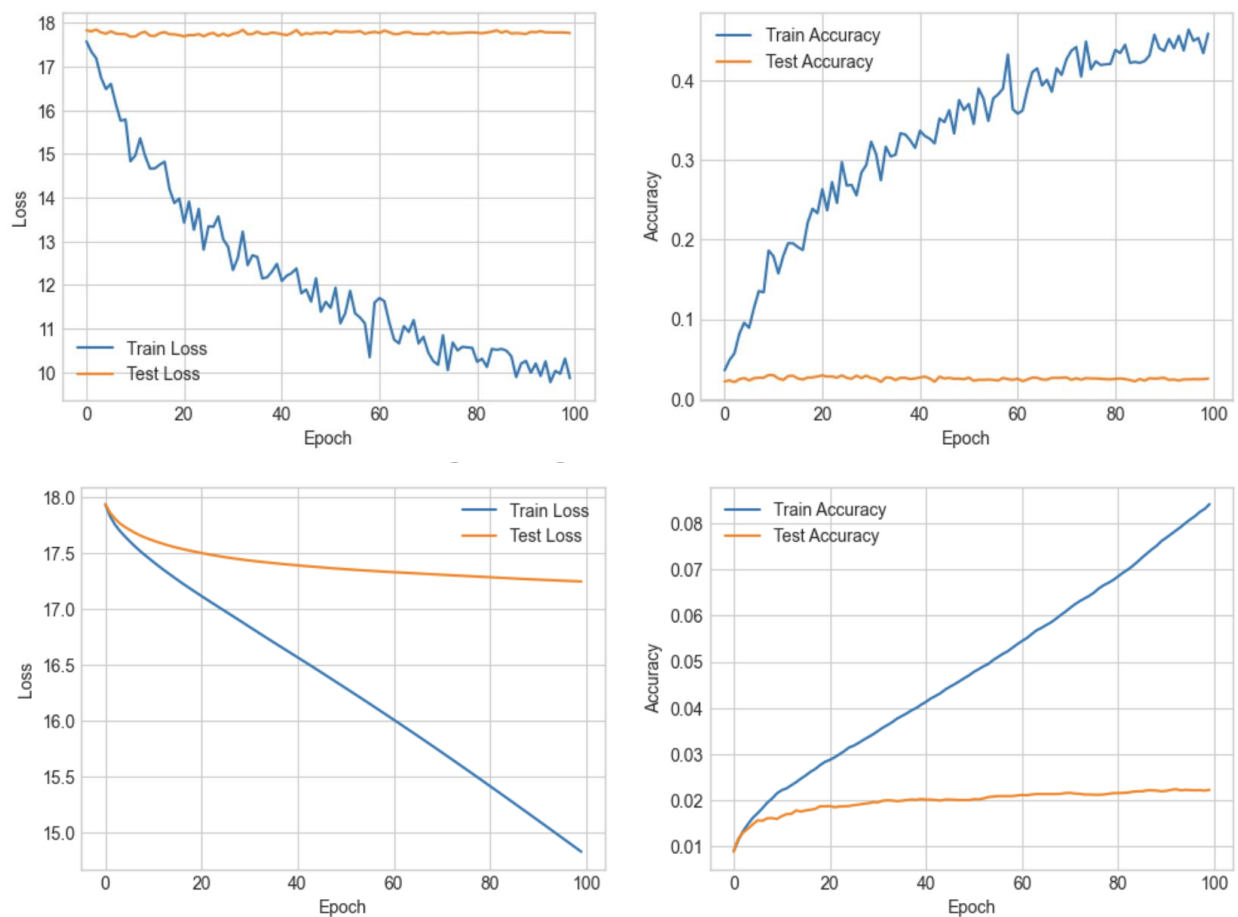


Figure 2. Top: Logistic regression with momentum, non-normalized inputs. Bottom: Logistic regression with momentum, normalized inputs

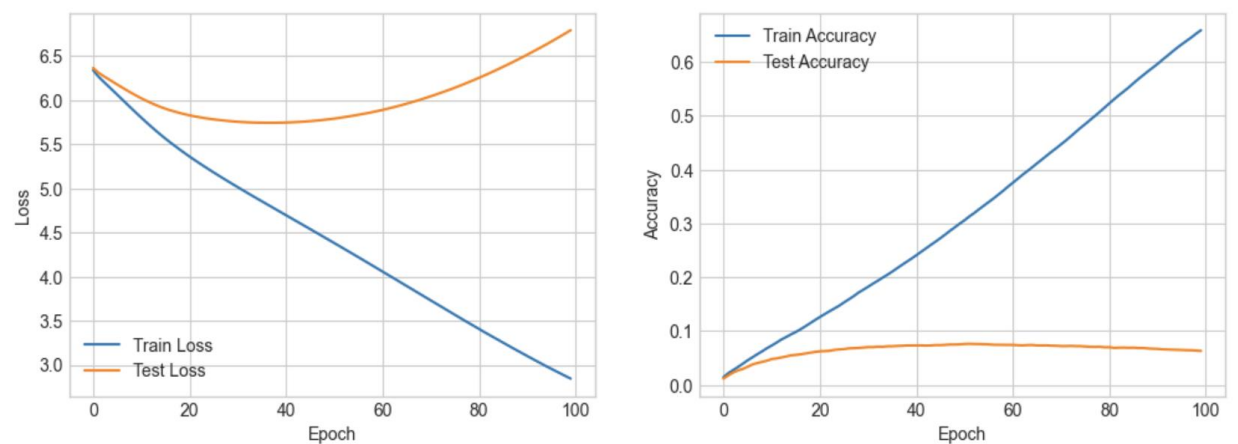


Figure 3: Three-layer neural net with L2 regularization, Adam, He initialization, normalized inputs, and softmax clipping

Summary of potential extra credit methods

1. **Zero-centered normalization:** helps with He initialization efficiency, works well with ReLU activation layers which are also zero-centered.
2. **Momentum:** helps avoid local minima by accounting for previous gradient weights.
3. **Clipped softmax activation function:** avoids underflow and overflow errors by limiting values that can be passed to the denominator of the softmax function.
4. **Adam:** improved momentum method that adapts learning rate based on individual parameters. It is both computationally more efficient and allows for more robust convergence.
5. **L2 (Ridge) Regularization:** shrinks the weights of the model to reduce overfitting on the training dataset.
6. **He Initialization:** initializes weights based on input size in order to keep variance stable for each activation throughout the network. Works well with zero-centered normalization, ReLU activation, and L2 regularization.

References

- [1] J. Brownlee, "How to Manually Scale Image Pixel Data for Deep Learning," *Machine Learning Mastery*, Jul. 05, 2019. <https://machinelearningmastery.com/how-to-manually-scale-image-pixel-data-for-deep-learning/>
- [2] J. Brownlee, "How to Normalize, Center, and Standardize Image Pixels in Keras," *Machine Learning Mastery*, Apr. 03, 2019. <https://machinelearningmastery.com/how-to-normalize-center-and-standardize-images-with-the-imagedatagenerator-in-keras/>
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," in *2015 IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile: IEEE, Dec. 2015, pp. 1026–1034. doi: 10.1109/ICCV.2015.123.
- [4] V. Kakaraparthi, "Xavier and He Normal (He-et-al) Initialization," *Prateek Vishnu*, Sep. 29, 2018. <https://prateekvishnu.medium.com/xavier-and-he-normal-he-et-al-initialization-8e3d7a087528>
- [5] R. C. Gonzalez and R. E. Woods, *Digital image processing*, 3rd ed. Upper Saddle River, N.J: Prentice Hall, 2008.
- [6] Fei-Fei Li, Yunzhu Li, and R. Gao, "CS231n Convolutional Neural Networks for Visual Recognition." Stanford, Jan. 2023. [Online]. Available: <https://cs231n.github.io/neural-networks-2/#datapre>
- [7] R. Bhat, "Gradient Descent with Momentum," *Towards Data Science*, Oct. 03, 2020. <https://towardsdatascience.com/gradient-descent-with-momentum-59420f626c8f>
- [8] S. Park, "A 2021 Guide to Improving CNNs-Optimizers: Adam vs. SGD," *Geek Culture*, Jun. 20, 2021. <https://medium.com/geekculture/a-2021-guide-to-improving-cnns-optimizers-adam-vs-sgd-495848ac6008>
- [9] A. Oppermann, "Regularization in Deep Learning - L1, L2, and Dropout," *Towards Data Science*, Feb. 19, 2020. <https://towardsdatascience.com/regularization-in-deep-learning-l1-l2-and-dropout-377e75acc036>